Lecture 03.15: Practice Examples (Functions)

📌 Executive Summary

Modular decomposition: Writing pure functions, geometric calculations, validation routines, and composite pipelines.

🔬 Theoretical Foundations & Principles

1. Pure Functions & Side Effects

A function is considered **pure** if:

1. Given identical input arguments, it always returns identical outputs (deterministic).
2. It produces zero side effects (no mutation of non-local state, no I/O mutations).

Pure functions are dramatically easier to unit test, refactor, and parallelize across CPU cores.

💻 Production Implementation & Code

```
# Pure function: Temperature conversion
def celsius_to_fahrenheit(c: float) -> float:
    return (c * 9.0 / 5.0) + 32.0

def fahrenheit_to_celsius(f: float) -> float:
    return (f - 32.0) * 5.0 / 9.0

print("100 C in F:", celsius_to_fahrenheit(100))
print("212 F in C:", fahrenheit_to_celsius(212))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.